KANT AX/AI 엔지니어 트랙 · 11

터미널에서 GitHub 다루기

gh로 로그인하고
레포 만들고 지우기



오늘의 순서

1

설치하고 로그인하기

gh auth login, 권한 목록, git도 같이 로그인

5–14쪽

2

이미 있는 레포 연결하기

gh repo clone, view, browse, list

15–18쪽

3

새 레포 만들기

gh repo create 뒤 git push origin main

19–23쪽

4

레포 지우기와 권한 추가

delete가 막힐 때 gh auth refresh

24–29쪽

5

자주 쓰는 gh 명령

이슈, PR 만들기와 합치기

30–34쪽

3–4쪽은 들어가기, 35–36쪽은 막힐 때와 한 장 요약이에요.

들어가기

git, GitHub, gh는 서로 다른 도구예요

git

내 컴퓨터의 기록 도구

`add, commit`

`push, pull로 주고받기`

02부터 써 온 명령

GitHub 웹사이트

브라우저로 여는 저장소 집

`New repository 버튼`

`Pull request 버튼`

레포 만들기, PR은 여기서 했어요

gh

GitHub 버튼을 터미널에서 누르는 도구

`gh repo create`

`gh pr create`

오늘 배울 명령

gh는 git을 대신하지 않아요. 웹사이트에서 누르던 버튼을 맡아요.

들어가기

웹에서 누르던 버튼과 gh 명령

로그인	`gh auth login`
저장소 받기 (Code › 주소 복사)	`gh repo clone kim/team-intro`
New repository	`gh repo create my-notes --public`
Settings › Delete this repository	`gh repo delete kim/my-notes`
New issue	`gh issue create`
Compare & pull request	`gh pr create`

오늘은 이 여섯 줄을 위에서부터 차례로 쳐 봐요.

1. 설치하고 로그인하기

gh 설치하기

Mac

```
brew install gh
```

09에서 uv를 설치한 Homebrew로 설치해요.

Windows

```
winget install --id GitHub.cli -e
```

설치가 끝나면 Git Bash를 껐다 다시 열어요.

설치됐는지 확인

```
~ $ gh --version
gh version 2.89.0 (2026-03-26)
https://github.com/cli/cli/releases/tag/v2.89.0
```

버전이 나오면 설치 끝이에요. 숫자는 설치한 날에 따라 달라요.

1. 설치하고 로그인하기

gh auth status · 로그인했는지 묻기

```
~ $ gh auth status
You are not logged into any GitHub hosts. To log in, run: gh auth login
```

아직 로그인하지 않았어요

gh는 내가 누구인지 모르면 GitHub에 아무것도 요청할 수 없어요. 출력 끝에 할 일이 적혀 있어요. gh auth login이에요.

auth
로그인 관련

status
상태 보기

gh 다음에 대상, 그다음에 할 일을 적어요.

질문 1 · 어디서 쓰나요?

```
~ $ gh auth login
? Where do you use GitHub?  [Use arrows to move, type to filter]
> GitHub.com
  Other
```

질문 1 / 4

고를 것

GitHub.com

누를 키 · Enter

github.com 웹사이트를 쓰니 그대로 Enter예요. Other는 회사 전용 GitHub일 때 골라요.

질문 2 · git 주고받는 방식

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host?  [Use arrows to move, type to filter]
> HTTPS
  SSH
```

질문 2 / 4

고를 것

HTTPS

누를 키 · Enter

02부터 써 온 https:// 주소 방식이에요. SSH는 따로 열쇠 파일을 만들어야 해서 이 과정에서는 쓰지 않아요.

질문 3 · git도 같이 로그인할까요?

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? (Y/n)
```


고를 것

Yes

누를 키 · Y 또는 Enter

Yes로 하면 git push 할 때도 gh 로그인을 써요. 비밀번호나 토큰 창이 더는 뜨지 않아요.

질문 4 · 어떻게 로그인할까요?

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI?  [Use arrows to move, type to filter]
> Login with a web browser
  Paste an authentication token
```

질문 4 / 4

고를 것

Login with a web browser

누를 키 · Enter

브라우저로 로그인해요. 토큰을 직접 만들어 붙여 넣는 방법은 쓰지 않아요.

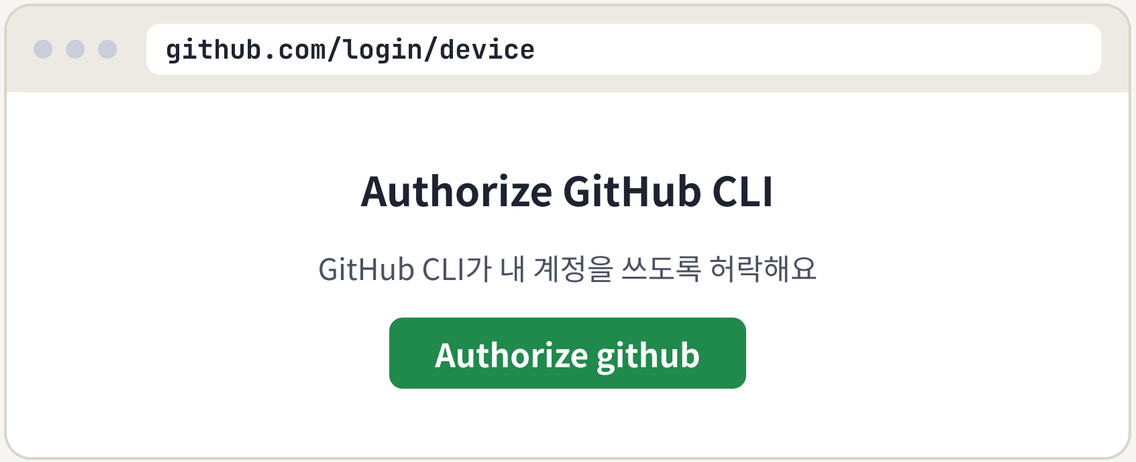
1. 설치하고 로그인하기

한 번만 쓰는 코드를 브라우저에 넣어요

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 8F2A-71C4 1
Press Enter to open https://github.com/login/device in your browser... 2
```





1. 설치하고 로그인하기

로그인 끝 · 터미널이 알려 주는 다섯 줄

```
~ $ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 8F2A-71C4
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete. (1)
- gh config set -h github.com git_protocol https
✓ Configured git protocol (2)
! Authentication credentials saved in plain text
✓ Logged in as kim (3)
```

1. 브라우저에서 허락 완료
2. git 주소 방식을 HTTPS로 저장
3. kim 계정으로 로그인

1. 설치하고 로그인하기

다시 gh auth status · 토큰과 권한 목록

```
~ $ gh auth status
github.com
  ✓ Logged in to github.com account kim (/home/kim/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_********* 1
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow' 2
```

1 · 토큰 = 출입증

로그인하면 GitHub이 gh에게 출입증(토큰)을 줘요. 화면에는 앞 네 글자만 보이고 나머지는 *로 가려요.

2 · 권한(scope) = 출입증으로 열 수 있는 문

repo는 레포 읽고 쓰기, read:org는 조직 보기, gist는 메모 조각, workflow는 Actions 설정이에요. 이 목록에 없는 일은 못 해요.

2번 목록을 기억해 둬요. 4파트에서 여기에 한 줄이 늘어요.

1. 설치하고 로그인하기

질문 3의 Yes가 바꾼 것 · git도 gh로 로그인해요

```
~ $ cat ~/.gitconfig
…
[pull]
    rebase = false
[credential "https://github.com"]
    helper =
    helper = !/usr/local/bin/gh auth git-credential
…
```

`git push origin main`

↓

git이 gh에게 출입증을 물어요

↓

GitHub에 바로 올라가요

02의 첫 push에서 뜨던 로그인 창, 토큰 입력 창이 이제 안 떠요.

이 줄은 gh가 넣었어요. 직접 고치지 않아요.

2. 이미 있는 레포 연결하기

gh repo clone · 주소 대신 주인/이름

git으로 받을 때 (02)

```
~/kant $ git clone https://github.com/kim/team-intro.git
```

gh로 받을 때

```
~/kant $ gh repo clone kim/team-intro
```

```
~/kant $ gh repo clone kim/team-intro
Cloning into 'team-intro'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (6/6), 510 bytes | 510.00 KiB/s, done.
```

받는 내용은 같아요. 주소 앞뒤(https://github.com/, .git)를 안 적어도 돼요.

2. 이미 있는 레포 연결하기

받은 폴더는 git clone과 똑같아요

```
~/kant $ cd team-intro
~/kant/team-intro $ git remote -v
origin    https://github.com/kim/team-intro.git (fetch)
origin    https://github.com/kim/team-intro.git (push)
~/kant/team-intro $ git log --oneline
1baff03 (HEAD -> main, origin/main, origin/HEAD) feat: kim 소개 추가
a2a187e docs: README 추가
```

origin도 그대로

gh가 origin 이름으로 https 주소를 등록했어요. git push origin main, git pull origin main을 지금까지처럼 쳐요.

git으로 받은 폴더도 OK

06에서 git clone으로 받은 폴더에서도 gh를 바로 쓸 수 있어요. gh는 origin 주소를 보고 어느 레포인지 알아요.

2. 이미 있는 레포 연결하기

레포 폴더 안에서 gh repo view, gh browse

```
~/kant/team-intro $ gh repo view
kim/team-intro
No description provided


  # team-intro

  우리 팀 소개 페이지예요.


View this repository on GitHub: https://github.com/kim/team-intro
```

```
~/kant/team-intro $ gh browse
Opening https://github.com/kim/team-intro in your browser.
```

gh repo view

레포 이름과 README를 터미널에 보여 줘요.

gh browse

지금 레포를 브라우저로 열어요.

주인/이름을 안 적어도 돼요. 지금 폴더의 레포를 써요.

2. 이미 있는 레포 연결하기

gh repo list · 내 레포 목록

```
~ $ gh repo list

Showing 2 of 2 repositories in @kim

NAME             DESCRIPTION  INFO    UPDATED
kim/hello-git                public  about 3 minutes ago
kim/team-intro               public  about 3 minutes ago
```

어디서 쳐도 돼요

내 계정의 레포 목록이라 레포 폴더 밖, 홈 폴더에서 쳐도 같아요.

이름을 확인하는 곳

clone이나 delete에 적을 주인/이름을 여기서 복사해요.

3. 새 레포 만들기

웹에서 만들 때와 gh로 만들 때

웹에서 (02)

1. GitHub에서 New 누르기
2. 이름 적고 Create
3. 주소 복사
4. `git remote add origin 주소`

gh로

```
gh repo create my-notes
  --public --source=. --remote=origin
```

브라우저 없이 한 줄로 레포를 만들고 origin까지 연결해요.

만들고 연결하는 데까지 gh가 해요. push는 지금처럼 git으로 해요.

3. 새 레포 만들기

먼저 내 컴퓨터에 저장소와 커밋을 만들어요

```
~/kant $ mkdir my-notes
~/kant $ cd my-notes
~/kant/my-notes $ git init
Initialized empty Git repository in /home/kim/kant/my-notes/.git/
~/kant/my-notes $ echo '# my-notes' > README.md
~/kant/my-notes $ git add README.md
~/kant/my-notes $ git commit -m 'docs: README 추가'
[main (root-commit) f5c3a4c] docs: README 추가
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

02에서 한 순서 그대로예요. 커밋이 하나 있어야 GitHub에 올릴 게 생겨요.

3. 새 레포 만들기

gh repo create · 만들고 origin 연결까지

```
~/kant/my-notes $ gh repo create my-notes --public --source=. --remote=origin
✓ Created repository kim/my-notes on github.com 1
  https://github.com/kim/my-notes
✓ Added remote https://github.com/kim/my-notes.git 2
```

my-notes

GitHub에 만들 레포 이름

--public

누구나 볼 수 있게. 나만 보려면 --private

--source=.

지금 폴더(.)를 레포로 써요

--remote=origin

origin 이름으로 연결해요

--push 옵션은 붙이지 않아요. push는 다음 장에서 git push origin main으로 직접 쳐요.

3. 새 레포 만들기

push는 git으로 · 그리고 목록 확인

```
~/kant/my-notes $ git remote -v
origin    https://github.com/kim/my-notes.git (fetch)
origin    https://github.com/kim/my-notes.git (push)
~/kant/my-notes $ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 230 bytes | 230.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kim/my-notes.git
 * [new branch]      main -> main
```

```
~ $ gh repo list

Showing 3 of 3 repositories in @kim

NAME             DESCRIPTION  INFO    UPDATED
kim/hello-git
kim/my-notes
kim/team-intro
```

로그인 창 없이 push가 끝났어요. 질문 3의 Yes 덕분이에요.

3. 새 레포 만들기

옵션을 잊었으면 gh repo create만 쳐요

```
~/kant $ gh repo create
? What would you like to do?  [Use arrows to move, type to filter]
> Create a new repository on github.com from scratch
  Create a new repository on github.com from a template repository
  Push an existing local repository to github.com
```

빈 레포를 새로

Create a new repository … from scratch

템플릿에서

… from a template repository

내 폴더를 올리기

Push an existing local repository …

질문에 답하면 앞 장의 옵션을 gh가 채워요. 화살표와 Enter로 골라요.

4. 레포 지우기와 권한 추가

지우기 전에 · 되돌릴 수 없어요

사라져요

GitHub의 레포

커밋 기록, README

사라져요

이슈와 PR

토론과 리뷰 기록

사라져요

스타와 포크 연결

다른 사람이 누른 것

남아요

내 컴퓨터 폴더

~/kant/my-notes는 남아요

휴지통이 없어요. 지운 레포는 다시 꺼낼 수 없어요.

연습용으로 만든 my-notes만 지워요. team-intro처럼 팀이 쓰는 레포는 지우지 않아요.

4. 레포 지우기와 권한 추가

gh repo delete · 첫 시도는 막혀요

```
~ $ gh repo delete kim/my-notes

? Type kim/my-notes to confirm deletion: kim/my-notes 1
HTTP 403: Must have admin rights to Repository. (https://api.github.com/repos/kim/my-notes)
This API operation needs the "delete_repo" scope. To request it, run: 2
  gh auth refresh -h github.com -s delete_repo
```

1 · 이름을 한 번 더 쳐요

실수로 지우지 않게 주인/이름을 그대로 다시 쳐야 해요.

2 · delete_repo 권한이 없어요

출입증에 레포 지우기 권한이 없어서 GitHub이 거절했어요(403). 칠 명령까지 알려 줘요.

고장이 아니에요. 처음 로그인할 땐 지우기 권한을 일부러 안 받아요.

4. 레포 지우기와 권한 추가

출입증에 지우기 권한을 한 칸 더해요

지금 출입증 (토큰)

`repo` `read:org` `gist` `workflow` `delete_repo ?`

1파트 gh auth status의 Token scopes 줄이에요.

`gh auth refresh`

출입증을 새로 받아요

`-h github.com`

어느 GitHub인지

`-s delete_repo`

더할 권한(scope)

refresh는 질문 네 개 없이 출입증만 다시 받아요.

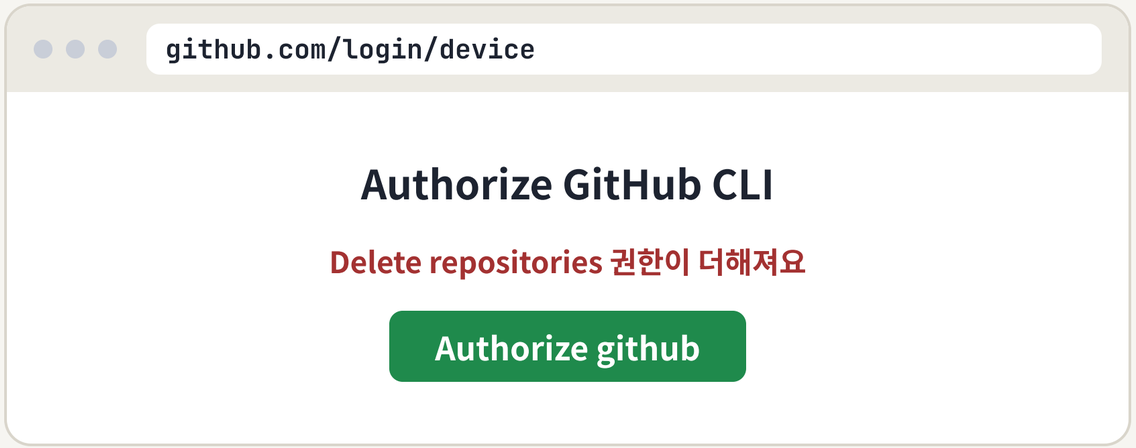
4. 레포 지우기와 권한 추가

gh auth refresh · 코드를 한 번 더 넣어요

```
~ $ gh auth refresh -h github.com -s delete_repo

! First copy your one-time code: 8F2A-71C4 1
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete. 2
```





4. 레포 지우기와 권한 추가

권한 목록에 delete_repo가 생겼어요

```
~ $ gh auth status
github.com
  ✓ Logged in to github.com account kim (/home/kim/.config/gh/hosts.yml)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_*********
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
  - Token scopes: 'delete_repo', 'gist', 'read:org', 'repo', 'workflow'
```

위 회색 줄은 refresh 전, 노란 줄이 지금이에요.

`repo` `read:org` `gist` `workflow` `delete_repo`

이제 지우기를 다시 해 봐요.

4. 레포 지우기와 권한 추가

두 번째 gh repo delete · 지워졌어요

```
~ $ gh repo delete kim/my-notes

? Type kim/my-notes to confirm deletion: kim/my-notes
✓ Deleted repository kim/my-notes
~ $ gh repo list

Showing 2 of 2 repositories in @kim

NAME            DESCRIPTION  INFO    UPDATED
kim/hello-git                public  about 3 minutes ago
kim/team-intro               public  about 3 minutes ago
```

```
~/kant/my-notes $ git push origin main
fatal: repository 'https://github.com/kim/my-notes.git/' not found
```

내 폴더는 그대로

GitHub 쪽이 없어서 push하면 not found가 나요. 폴더가 필요 없으면 01의 rm -r로 지워요.

지우기는 이름 확인, 권한 확인 두 번을 거쳐요.

5. 자주 쓰는 gh 명령

gh issue · 할 일을 이슈로 남기기

```
~/kant/team-intro $ gh issue create --title "소개 페이지에 사진 넣기" --body "각자 사진 한 장씩 넣어요"

Creating issue in kim/team-intro

https://github.com/kim/team-intro/issues/1
```

```
~/kant/team-intro $ gh issue list

Showing 1 of 1 open issue in kim/team-intro

ID  TITLE                    LABELS  UPDATED
#1  소개 페이지에 사진 넣기          less than a minute ago
```

--title은 제목, --body는 내용이에요. 빼면 gh가 하나씩 물어봐요.

5. 자주 쓰는 gh 명령

gh pr create · 브랜치 push 뒤 PR 만들기

```
~/kant/team-intro $ git checkout -b feature/kim-hobby
Switched to a new branch 'feature/kim-hobby'
~/kant/team-intro $ echo '취미: 등산' >> kim.md
~/kant/team-intro $ git add kim.md
~/kant/team-intro $ git commit -m 'feat: kim 취미 추가'
[feature/kim-hobby 8039736] feat: kim 취미 추가
 1 file changed, 1 insertion(+)
~/kant/team-intro $ git push origin feature/kim-hobby
…
 * [new branch]      feature/kim-hobby -> feature/kim-hobby
```

```
~/kant/team-intro $ gh pr create --title "feat: kim 취미 추가" --body "kim.md에 취미를 적었어요"

Creating pull request for feature/kim-hobby into main in kim/team-intro

https://github.com/kim/team-intro/pull/2
```

브랜치를 git push origin 브랜치로 먼저 올리고, 그다음 gh pr create예요.

5. 자주 쓰는 gh 명령

gh pr list, gh pr view · PR 보기

```
~/kant/team-intro $ gh pr list

Showing 1 of 1 open pull request in kim/team-intro

ID  TITLE                 BRANCH              CREATED AT
#2  feat: kim 취미 추가  feature/kim-hobby  less than a minute ago
```

```
~/kant/team-intro $ gh pr view 2
feat: kim 취미 추가 kim/team-intro#2
Open • kim (kim) wants to merge 1 commit into main from feature/kim-hobby • less than a minute ago
+1 -0 • No checks


  kim.md에 취미를 적었어요


View this pull request on GitHub: https://github.com/kim/team-intro/pull/2
```

5. 자주 쓰는 gh 명령

gh pr merge · 합치고 main 받아 오기

```
~/kant/team-intro $ gh pr merge 2 --merge
✓ Merged pull request kim/team-intro#2 (feat: kim 취미 추가)
~/kant/team-intro $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
~/kant/team-intro $ git pull origin main
…
   1baff03..5487d25  main       -> origin/main
Updating 1baff03..5487d25
Fast-forward
```

```
~/kant/team-intro $ git log --oneline
5487d25 Merge pull request #2 from kim/feature/kim-hobby
8039736 feat: kim 취미 추가
1baff03 feat: kim 소개 추가
a2a187e docs: README 추가
```

--merge = Create a merge commit

08에서 본 PR Merge 버튼 세 가지 중 첫째예요. squash, rebase 옵션도 있으니 팀이 정한 방식을 써요.

합친 뒤에는 지금까지처럼 git pull origin main으로 받아 와요.

5. 자주 쓰는 gh 명령

다른 명령은 gh --help에서 찾아요

```
~ $ gh --help
Work seamlessly with GitHub from the command line.

USAGE
  gh <command> <subcommand> [flags]

CORE COMMANDS
  auth:          Authenticate gh and git with GitHub
  browse:        Open repositories, issues, pull requests, and more in the browser
  codespace:     Connect to and manage codespaces
  gist:          Manage gists
  issue:         Manage issues
  org:           Manage organizations
  pr:            Manage pull requests
  project:       Work with GitHub Projects.
  release:       Manage releases
  repo:          Manage repositories
  …
```

오늘 쓴 묶음

`auth browse`

`issue pr repo`

한 단계 더 보기

`gh repo --help`

`gh pr create --help`

정리

gh에서 막히면

1	`gh: command not found` 설치 직후 터미널이 gh를 몰라요	`터미널 껐다 켜기`
2	`You are not logged into any GitHub hosts` 아직 로그인 전이에요	`gh auth login`
3	`needs the "delete_repo" scope` 지우기 권한이 없어요	`gh auth refresh -h github.com -s delete_repo`
4	`failed to run git: fatal: not a git repository` 레포 폴더 밖에서 쳤어요	`pwd 뒤 cd 레포 폴더`
5	**Where should we push the 'feature/...' branch?** 브랜치를 push하기 전에 PR을 만들었어요	`Ctrl+C 뒤 git push origin 브랜치`
6	**브라우저가 안 열려요** 주소를 직접 열어요	`github.com/login/device`

한 장 요약

로그인

`gh auth login`
로그인

`gh auth status`
로그인 상태, 권한

`gh auth refresh -s 권한`
권한 더하기

레포

`gh repo clone 주인/이름`
받기

`gh repo view`
지금 레포 보기

`gh repo list`
내 레포 목록

`gh repo create 이름 …`
만들기

`gh repo delete 주인/이름`
지우기

이슈와 PR

`gh issue create`
이슈 만들기

`gh issue list`
이슈 목록

`gh pr create`
PR 만들기

`gh pr list / view`
PR 보기

`gh pr merge 번호`
PR 합치기

커밋과 push는 git으로, 웹 버튼은 gh로 해요.
레포 지우기는 되돌릴 수 없어요. 지우기 전에 주인/이름을 한 번 더 확인해요.